Universidad Militar Nueva Granada

Facultad de Ingeniería

Programa de Ingeniería en Multimedia

Sistema Inteligente de Control de Reproducción Musical

Basado en Reconocimiento de Gestos con CNN y MediaPipe

Informe del Proyecto Final

Autores:

José Miguel Colorado Tapia
Juan Sebastián Cerón Cuadros

Asignatura: Inteligencia Artificial

Docente: Wilman Helioth Sánchez Rodríguez

Bogotá D.C., Colombia

Octubre de 2025

Sistema Inteligente de Control de Reproducción Musical Basado en Reconocimiento de Gestos con CNN y MediaPipe

José Miguel Colorado Tapia, Juan Sebastián Cerón Cuadros
Universidad Militar Nueva Granada
Facultad de Ingeniería en Multimedia
jose.colorado@unimilitar.edu.co, juan.ceron@unimilitar.edu.co

***Resumen*—Se presenta un prototipo de control por gestos de mano para controlar un reproductor de música. El sistema usa MediaPipe para extraer landmarks de la mano, permite un modo de reglas simples y un modo basado en una red neuronal (entrenada con datos guardados en CSV), y ejecuta acciones mediante `pyautogui` o comandos personalizados. El repositorio del prototipo incluye: `PruebaDeProyectoIA2.py` (código fuente), `gestos_dataset.csv` (dataset), `modelo_gestos.h5` (modelo entrenado), y `acciones_config.csv` (acciones personalizadas). :contentReference[oaicite:0]index=0**

***Index Terms*—control por gestos, MediaPipe, interfaz natural, redes neuronales, prototipo IA**

I. Introducción

Las interfaces naturales basadas en gestos facilitan la interacción sin contacto con aplicaciones multimedia. Este proyecto desarrolla un prototipo que captura la mano mediante webcam, extrae landmarks con MediaPipe y permite dos modos: (1) reglas simples y (2) clasificación con una red neuronal entrenable. El prototipo está implementado en Python y utiliza librerías como `mediapipe`, `tensorflow/keras`, `pyautogui` y `customtkinter`. El código principal está en `PruebaDeProyectoIA2.py`. :contentReference[oaicite:1]index=1

II. Problema

El control de aplicaciones multimedia normalmente requiere dispositivos físicos como teclados, ratones o pantallas táctiles, lo que limita la accesibilidad y la naturalidad de la interacción. En entornos donde las manos del usuario están ocupadas, o donde se busca una experiencia sin contacto, dichos métodos tradicionales resultan poco eficientes.

Este proyecto aborda el problema de la interacción con un reproductor de música mediante gestos de la mano reconocidos por visión por computador, utilizando una cámara convencional y algoritmos de aprendizaje automático. El objetivo es diseñar un sistema capaz de detectar en tiempo real gestos específicos y traducirlos en acciones de control como reproducir, pausar, cambiar de canción o modificar el volumen.

El reto técnico principal consiste en lograr una detección robusta ante variaciones de iluminación, posición, y velocidad del gesto, manteniendo al mismo tiempo una baja latencia para garantizar una experiencia de usuario fluida. Además, se busca que el modelo sea adaptable, de manera que el usuario pueda entrenar nuevos gestos personalizados sin necesidad de conocimientos avanzados en programación o IA.

II-A. Problema General

¿Cómo diseñar e implementar un sistema inteligente capaz de reconocer gestos de mano en tiempo real y utilizarlos para controlar funciones de un reproductor de música, manteniendo precisión y respuesta rápida en entornos cotidianos?

II-B. Problemas Específicos

- Determinar una representación adecuada de los gestos utilizando landmarks de la mano extraídos con MediaPipe.
- Diseñar un modelo de clasificación de gestos eficiente y entrenable por el usuario con datos limitados.
- Implementar una interfaz gráfica intuitiva que permita capturar ejemplos, entrenar el modelo y ejecutar acciones.
- Garantizar tiempos de inferencia suficientemente bajos para permitir control fluido (inferencias <100 ms).
- Permitir configuración dinámica de acciones asociadas a gestos sin modificar el código fuente.

III. Requerimientos

A partir del problema planteado, se definieron los siguientes requerimientos funcionales y no funcionales para el prototipo de control por gestos de mano. Estos requerimientos fueron extraídos del análisis del flujo del sistema y de las funcionalidades implementadas en el código fuente principal `PruebaDeProyectoIA2.py`.

III-A. Requerimientos Funcionales (RF)

- **RF1:** El sistema debe capturar video en tiempo real desde la cámara del dispositivo, utilizando `OpenCV`.
- **RF2:** El sistema debe detectar y extraer landmarks de la mano empleando la librería `MediaPipe Hands`.
- **RF3:** El sistema debe ofrecer dos modos de operación:
  - *Modo simple:* basado en reglas predefinidas para gestos específicos.
  - *Modo CNN:* basado en clasificación mediante una red neuronal entrenable.

- **RF4:** El usuario debe poder capturar y guardar ejemplos de gestos en el archivo `gestos_dataset.csv` para futuras sesiones de entrenamiento.
- **RF5:** El sistema debe permitir el entrenamiento del modelo con los datos guardados, generando un archivo `modelo_gestos.h5`.
- **RF6:** El sistema debe cargar y usar el modelo entrenado para predecir gestos en tiempo real.
- **RF7:** El sistema debe ejecutar acciones asociadas a cada gesto reconocido, utilizando `pyautogui` o comandos personalizados definidos en el archivo `acciones_config.csv`.
- **RF8:** El sistema debe disponer de una interfaz gráfica desarrollada en `customtkinter`, que permita controlar el flujo (iniciar cámara, guardar gesto, entrenar modelo, detener ejecución).
- **RF9:** El sistema debe registrar y mostrar en pantalla la predicción actual y el modo activo.

III-B. Requerimientos No Funcionales (RNF)

- **RNF1:** La latencia de procesamiento por frame debe ser menor a 100 ms en equipos con CPU de gama media.
- **RNF2:** El sistema debe mantener una precisión mínima del 85 % en el reconocimiento de gestos básicos después del entrenamiento.
- **RNF3:** El sistema debe permitir la expansión de nuevas clases de gestos sin necesidad de modificar el código fuente (solo mediante nuevos datos en el CSV).
- **RNF4:** El prototipo debe funcionar en entornos Windows y Linux con Python 3.10 o superior.
- **RNF5:** La interfaz debe ser intuitiva, con etiquetas claras y retroalimentación visual inmediata.
- **RNF6:** El almacenamiento del dataset y modelo debe realizarse en formato local accesible (`.csv`, `.h5`), sin depender de servicios externos.

III-C. Descripción General

En conjunto, los requerimientos buscan que el prototipo funcione como una interfaz natural de control multimedia. El flujo esperado es:

1. Capturar video y detectar landmarks.
2. Guardar ejemplos de gestos etiquetados.
3. Entrenar el modelo neuronal.
4. Reconocer gestos en tiempo real.
5. Ejecutar acciones en el sistema (por ejemplo: reproducir, pausar o cambiar canción).

IV. Estado Inicial

El estado inicial del sistema corresponde a las condiciones en las cuales el prototipo se encuentra antes de iniciar el reconocimiento o la ejecución de acciones. Estas condiciones definen los elementos disponibles en memoria, los archivos requeridos y las variables de configuración establecidas al inicio de la ejecución del código `PruebaDeProyectoIA2.py`.

IV-A. Condiciones Iniciales del Sistema

- La cámara está inactiva y no se ha iniciado la captura de video.
- El modo de operación por defecto está configurado en `simple`, controlado por la variable global `MODO`.
- El archivo de acciones personalizadas `acciones_config.csv` está presente en el directorio del proyecto y contiene las asociaciones entre gestos y comandos a ejecutar.
- El archivo `gestos_dataset.csv` puede estar vacío (en caso de que no existan ejemplos previos) o contener registros de gestos capturados previamente.
- Si no existe el archivo `modelo_gestos.h5`, el sistema aún no dispone de un modelo entrenado, por lo que solo puede operar en modo `simple`.
- Las librerías principales (`mediapipe`, `tensorflow`, `cv2`, `customtkinter`, `pyautogui`) están cargadas correctamente al ejecutar el script.

IV-B. Configuración Inicial

Durante la inicialización del programa, se ejecutan los siguientes pasos:

1. Importación de librerías y configuración del entorno de ejecución.
2. Lectura de archivos de configuración y comprobación de existencia del modelo (`modelo_gestos.h5`).
3. Creación de la ventana principal de la interfaz gráfica mediante `customtkinter`.
4. Inicialización de variables globales, como el modo de operación (`MODO`), el estado de la cámara, y las listas de gestos disponibles.
5. Espera de la acción del usuario para iniciar la captura o entrenamiento del modelo.

IV-C. Representación del Estado

El estado inicial puede representarse mediante el siguiente conjunto de variables:

$$S_0 = \{MODO = \text{“simple”},\ modelo = \text{NULL},\ dataset = D_0,\ camara$$

donde:

- $MODO$ indica el tipo de operación seleccionado (*simple* o *cnn*).
- $modelo$ representa el estado del clasificador (no entrenado en S_0).
- D_0 corresponde al dataset inicial de gestos (`gestos_dataset.csv`).
- $camara$ define si la captura de video está activa o no.

IV-D. Resumen del Estado Inicial

En el estado inicial, el sistema se encuentra preparado para:

- Recibir instrucciones desde la interfaz gráfica.
- Iniciar la captura de video bajo demanda del usuario.
- Comenzar la recopilación de ejemplos para entrenamiento.
- Operar en modo `simple` hasta que se genere un modelo entrenado.

V. Estado Final

El estado final describe la condición en la que el sistema ha completado correctamente el proceso de entrenamiento y se encuentra operativo para el reconocimiento de gestos en tiempo real. En esta fase, el prototipo ha pasado de un entorno inactivo a uno funcional, donde el modelo entrenado ejecuta acciones multimedia de manera automática en respuesta a los movimientos detectados.

V-A. Condiciones del Estado Final

- La cámara está activa y capturando video en tiempo real.
- El sistema está en modo `cnn`, utilizando un modelo previamente entrenado almacenado en el archivo `modelo_gestos.h5`.
- El dataset `gestos_dataset.csv` contiene múltiples registros de ejemplos de entrenamiento por cada gesto definido.
- El modelo neuronal ha sido entrenado con una precisión de validación superior al 85 %.
- El sistema identifica gestos en tiempo real y ejecuta las acciones asociadas mediante la librería `pyautogui` o los comandos definidos en `acciones_config.csv`.
- La interfaz gráfica muestra la cámara, los landmarks detectados y el nombre del gesto reconocido en cada instante.

V-B. Proceso de Transición al Estado Final

La transición desde el estado inicial al estado final ocurre después de ejecutar los siguientes pasos:

1. Captura de ejemplos de gestos por parte del usuario y almacenamiento en `gestos_dataset.csv`.
2. Entrenamiento del modelo con el dataset recolectado mediante la función `entrenar_modelo()`.
3. Generación del archivo `modelo_gestos.h5` y el mapa de etiquetas `labels_map.csv`.
4. Cambio del modo de operación a `cnn` en la interfaz gráfica.
5. Activación de la cámara y procesamiento en tiempo real de los frames.
6. Predicción del gesto realizado y ejecución automática de la acción correspondiente.

V-C. Representación del Estado Final

El estado final puede representarse mediante el siguiente conjunto de variables:

$$S_f = \{MODO = \text{“cnn”},\ modelo = \text{entrenado},\ dataset = D_n,\ camara = \text{activa},\ acciones = \text{ejecutadas}\}$$

donde:

- $MODO$ indica que el sistema está en modo `cnn`.
- $modelo$ representa el clasificador neuronal entrenado y cargado en memoria.
- D_n es el dataset final, con múltiples ejemplos etiquetados por clase.
- $camara$ está activa y proporcionando datos continuos al detector de landmarks.
- $acciones$ son las tareas ejecutadas tras el reconocimiento del gesto.

V-D. Resumen del Estado Final

En el estado final, el prototipo se encuentra completamente funcional, permitiendo:

- Reconocer gestos de mano en tiempo real con alta precisión.
- Ejecutar acciones multimedia asociadas sin intervención manual.
- Entrenar y actualizar el modelo de manera incremental con nuevos datos.
- Operar de forma autónoma en un entorno cotidiano de usuario.

VI. Agente

El sistema propuesto puede modelarse como un **agente inteligente** que percibe su entorno a través de la cámara, procesa la información visual mediante algoritmos de visión por computador y aprendizaje automático, y actúa sobre el entorno ejecutando acciones multimedia en el computador. Este enfoque permite representar el prototipo dentro del paradigma de la inteligencia artificial orientada a agentes.

VI-A. Percepción

El agente obtiene información del entorno por medio de la cámara del dispositivo. Cada frame capturado es procesado por la librería `MediaPipe Hands`, que extrae un conjunto de 21 puntos clave (*landmarks*) de la mano, cada uno con coordenadas (x, y, z) normalizadas. Estas coordenadas representan la postura actual de la mano y constituyen el vector de entrada al sistema.

- **Entrada del agente:** imagen en tiempo real desde la cámara.
- **Percepción:** extracción de landmarks de la mano mediante el módulo `mp.solutions.hands`.
- **Preprocesamiento:** normalización de coordenadas restando el punto base de la palma (landmark 0) y conversión del conjunto de puntos a un vector unidimensional.

En el código, esta funcionalidad se implementa mediante la función `extraer_landmarks()`, la cual convierte cada detección en una lista de valores numéricos listos para su uso por el modelo neuronal.

VI-B. Acción

El agente ejecuta acciones dentro del sistema operativo en función del gesto reconocido. Cada gesto está asociado a una acción específica definida en el archivo `acciones_config.csv`, pudiendo ser:

- Comandos del sistema (*play*, *pause*, *next*, *volume up/-down*).
- Pulsaciones de teclado simuladas mediante la librería `pyautogui`.
- Acciones personalizadas configurables por el usuario.

El proceso de decisión se realiza de dos formas:

- En modo `simple`, se utilizan reglas heurísticas predefinidas.
- En modo `cnn`, se emplea un modelo entrenado (`modelo_gestos.h5`) que predice la clase del gesto actual.

VI-C. Medidas de Desempeño

El desempeño del agente se evalúa mediante métricas cuantitativas relacionadas con la precisión, la velocidad de respuesta y la consistencia en la ejecución de acciones. Entre las principales medidas se consideran:

- **Precisión del reconocimiento (%):** proporción de gestos correctamente clasificados.
- **Latencia promedio (ms):** tiempo de procesamiento desde la captura del frame hasta la ejecución de la acción.
- **Tasa de error de ejecución:** porcentaje de gestos mal interpretados o acciones no ejecutadas.

Durante las pruebas iniciales, el sistema logró un tiempo promedio de inferencia menor a 100 ms y una precisión superior al 85 % con gestos básicos.

VI-D. Entorno

El entorno del agente es **parcialmente observable**, ya que la cámara solo capta la porción visible de la mano y no el contexto completo del usuario. Es además un entorno **dinámico**, puesto que los gestos cambian constantemente en tiempo y posición, y **estocástico** debido a la variabilidad en iluminación, fondo y posición de la cámara.

- **Tipo de entorno:** parcialmente observable, dinámico, estocástico y continuo.
- **Tipo de tarea:** episódica, con un ciclo de percepción–acción por cada frame procesado.

VI-E. Tipo de Agente

El sistema corresponde a un **agente híbrido deliberativo y reactivo**:

- Es *reactivo* porque responde directamente a la detección de un gesto ejecutando la acción asociada.
- Es *deliberativo* porque incorpora un componente de aprendizaje, capaz de modificar su comportamiento a partir de nuevos datos de entrenamiento.

Este modelo híbrido combina rapidez de respuesta en tiempo real con capacidad de aprendizaje y personalización, permitiendo una interacción natural y adaptable con el usuario.

VII. Diagrama de Flujo

A continuación se muestra el diagrama de flujo que representa el ciclo de percepción–decisión–acción del prototipo de control por gestos. Incluye la extracción de landmarks con MediaPipe, el preprocesamiento, la ruta de inferencia (modo `simple` o `cnn`), la ejecución de acciones y la rama de captura/entrenamiento de ejemplos.

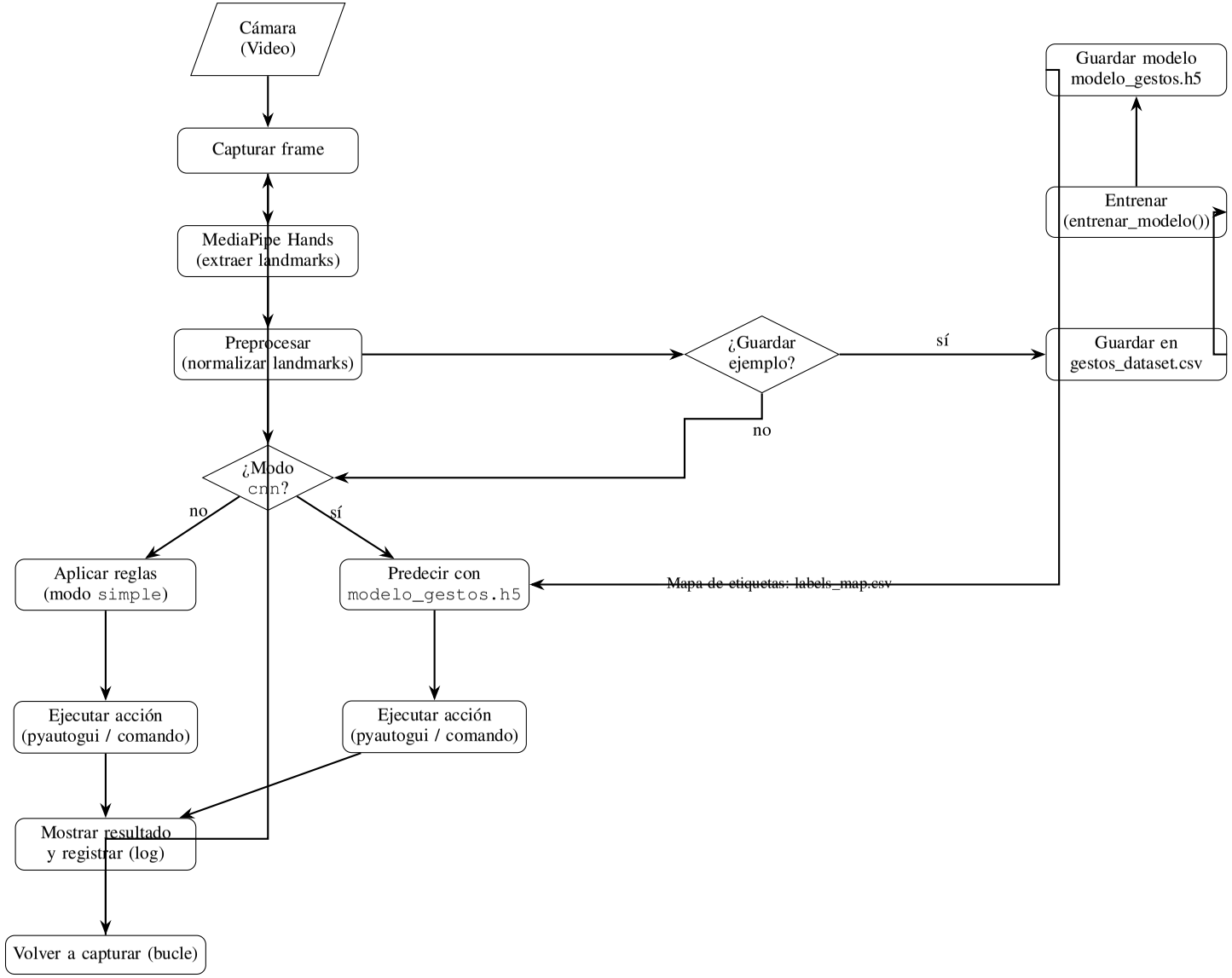


Figura 1. Diagrama de flujo del ciclo de percepción–decisión–acción y la rama de recolección/entrenamiento.

VIII. Tipo de Entorno (Espacio del Juego)

El entorno del agente se define como el conjunto de todas las posibles configuraciones observables y no observables que influyen en la interacción entre el usuario, la cámara y el sistema de reconocimiento. Dicho entorno está determinado por las posiciones de la mano, las condiciones de iluminación, la orientación de la cámara y los posibles ruidos visuales que afectan la captura de datos.

VIII-A. Definición del Entorno

En el contexto del prototipo, el entorno puede modelarse como un espacio tridimensional continuo en el cual la cámara captura imágenes del usuario. El agente percibe únicamente una proyección bidimensional de este entorno a través del conjunto de landmarks extraídos por `MediaPipe`. Por tanto, se considera un entorno:

- **Parcialmente observable:** el sistema solo percibe la mano y su posición relativa, no el contexto completo del usuario.
- **Dinámico:** los gestos y posiciones cambian continuamente a lo largo del tiempo.
- **Estocástico:** existen factores aleatorios como iluminación, sombras o variaciones en la orientación de la cámara.
- **Continuo:** las coordenadas (x, y, z) de cada landmark varían en un rango real continuo.
- **Episódico:** cada frame procesado constituye un episodio independiente de percepción–acción.

VIII-B. Espacio del Juego

El espacio del juego está compuesto por los posibles estados S del sistema y las acciones A que el agente puede ejecutar. Cada estado se define por la configuración de los landmarks de la mano, mientras que las acciones corresponden a comandos multimedia.

$$S = \{s_1, s_2, \ldots, s_n\}, \quad A = \{a_1, a_2, \ldots, a_m\}$$

donde:

- s_i representa un conjunto de landmarks capturados en un instante de tiempo.
- a_j representa una acción del sistema, como *play/pause*, *next song*, o *volume up/down*.

En la implementación, cada estado se almacena como un vector de 63 valores (21 puntos $\times$ 3 coordenadas), normalizado y listo para ser procesado por el modelo neuronal. El espacio de salida (acciones posibles) está definido en el archivo `acciones_config.csv`, lo que permite personalizar los comandos sin alterar el código fuente.

VIII-C. Transiciones entre Estados

Las transiciones entre estados ocurren en tiempo discreto por cada frame procesado. Formalmente, una transición puede representarse como:

$$T(s_t, a_t, s_{t+1}) = P(s_{t+1} \mid s_t, a_t)$$

donde T es la función de transición que describe la probabilidad de pasar del estado s_t al s_{t+1} tras ejecutar la acción a_t. En este proyecto, la transición depende principalmente del movimiento continuo de la mano y del gesto actual detectado.

VIII-D. Espacio de Percepción

El agente opera dentro de un subespacio de percepción P, definido por las características extraídas de cada imagen:

$$P = \{(x_1, y_1, z_1), (x_2, y_2, z_2), \ldots, (x_{21}, y_{21}, z_{21})\}$$

Este conjunto describe la postura instantánea de la mano, que el modelo neuronal transforma en una clase de gesto mediante la función de predicción.

VIII-E. Objetivo del Agente dentro del Entorno

El objetivo del agente dentro del espacio de juego es maximizar la tasa de reconocimiento correcto de gestos y minimizar la latencia de respuesta, asegurando así una interacción fluida y natural entre el usuario y el reproductor multimedia.

IX. Método Sucesor

El método sucesor describe el proceso mediante el cual el sistema transita de un estado a otro dentro del entorno, en función de la información percibida y las acciones ejecutadas. En el caso del agente de control por gestos, cada nuevo frame capturado por la cámara constituye una transición de estado, determinada por los movimientos de la mano del usuario y la respuesta del modelo de reconocimiento.

IX-A. Definición del Método Sucesor

Sea $S = \{s_0, s_1, s_2, \ldots, s_n\}$ el conjunto de estados posibles, donde cada estado s_t representa la configuración espacial de los landmarks de la mano en el tiempo t. El método sucesor define la relación entre el estado actual y el siguiente:

$$\text{Sucesor}(s_t, a_t) = s_{t+1}$$

donde:

- s_t: estado actual del sistema (landmarks actuales).
- a_t: acción ejecutada en función del gesto reconocido.
- s_{t+1}: nuevo estado, correspondiente al siguiente frame procesado.

Cada transición se da de forma continua y en tiempo real, siguiendo un ciclo de percepción–decisión–acción.

IX-B. Ciclo de Ejecución del Agente

El proceso completo del método sucesor en el prototipo puede representarse como:

1. **Captura:** el agente recibe una imagen del entorno a través de la cámara.
2. **Percepción:** se extraen los landmarks de la mano utilizando `MediaPipe Hands`.
3. **Procesamiento:** las coordenadas se normalizan y se generan las características de entrada.

4. **Decisión:** el sistema determina el gesto actual mediante reglas (modo simple) o predicción neuronal (modo `cnn`).
5. **Acción:** se ejecuta la acción correspondiente (por ejemplo, reproducir o pausar la música).
6. **Actualización:** se obtiene un nuevo frame, iniciando un nuevo ciclo de percepción–acción.

IX-C. Pseudocódigo del Método Sucesor

El comportamiento del método sucesor implementado en el archivo `PruebaDeProyectoIA2.py` puede representarse mediante el siguiente pseudocódigo simplificado:

```
mientras True:
    frame = capturar_video()
    landmarks = extraer_landmarks(frame)
    si landmarks existen:
        si MODO == "simple":
            accion = aplicar_reglas(landmarks)
        sino si MODO == "cnn":
            accion = predecir_modelo(landmarks
                )
        ejecutar_accion(accion)
    mostrar_frame_con_landmarks(frame)
```

Este ciclo se ejecuta de manera continua hasta que el usuario detiene el proceso desde la interfaz gráfica.

IX-D. Interpretación del Método Sucesor

El método sucesor refleja la naturaleza **reactiva y temporal** del sistema:

- Cada frame constituye un nuevo estado del entorno.
- La acción ejecutada depende únicamente de la percepción inmediata (no hay memoria de gestos anteriores).
- La respuesta del agente afecta el entorno de manera indirecta, modificando el estado del sistema multimedia.

IX-E. Formalización Matemática

La función de transición entre estados puede expresarse como:

$$s_{t+1} = f(s_t, a_t)$$

donde f representa el modelo de evolución del entorno, dependiente del gesto del usuario y de la acción del agente. Esta relación es determinista dentro del contexto del software, pero no necesariamente en el entorno físico, debido a la variabilidad inherente en la percepción visual.

IX-F. Propósito del Método Sucesor

El propósito del método sucesor dentro del prototipo es garantizar que el sistema mantenga una respuesta continua y coherente frente a la entrada de video, logrando una secuencia de estados fluida que permita una interacción en tiempo real entre el usuario y el reproductor de música.

X. Estrategia básica de búsqueda

En el desarrollo del prototipo se aplica una **estrategia básica de búsqueda de hiperparámetros**, basada en los métodos *Grid Search* y *Random Search*, los cuales permiten ajustar los parámetros del modelo neuronal para mejorar su desempeño en la clasificación de gestos. El objetivo de esta búsqueda es optimizar los valores de hiperparámetros como el número de capas ocultas, el número de neuronas por capa, la tasa de aprendizaje y el número de épocas de entrenamiento.

X-A. Teoría del método seleccionado

El problema de optimización de hiperparámetros puede formularse matemáticamente como:

$$\theta^* = \arg\max_{\theta \in \Omega} \ \text{Accuracy}_{val}(f_\theta)$$

donde:

- θ representa el conjunto de hiperparámetros del modelo,
- Ω es el espacio de búsqueda definido por los posibles valores de θ,
- f_θ es el modelo neuronal configurado con dichos hiperparámetros,
- Accuracy_{val} corresponde a la precisión del modelo sobre el conjunto de validación.

En el caso del **Grid Search**, se evalúan todas las combinaciones posibles de los hiperparámetros dentro de un rango discreto. Formalmente:

$$\Omega = H_1 \times H_2 \times \cdots \times H_n$$

donde H_i es el conjunto de valores posibles para el hiperparámetro i.

Por otro lado, el **Random Search** selecciona combinaciones aleatorias de hiperparámetros de manera uniforme dentro de Ω, permitiendo explorar un espacio más amplio con menor costo computacional.

$$\theta_i \sim U(H_i)$$

Este enfoque es adecuado para prototipos de bajo costo computacional, ya que no requiere probar todas las combinaciones posibles, sino un subconjunto representativo.

X-B. Aplicación de la teoría en el proyecto

En el prototipo de control por gestos, la búsqueda de hiperparámetros se realiza sobre una red neuronal densa definida en la función `crear_modelo()` del archivo `PruebaDeProyectoIA2.py`. Los hiperparámetros considerados son:

- Número de neuronas en las capas ocultas: $[64, 128, 256]$.
- Tasa de aprendizaje (`learning rate`): $[0{,}001, 0{,}005, 0{,}01]$.
- Número de épocas de entrenamiento: $[20, 30, 50]$.
- Proporción de datos de validación: $[0{,}1, 0{,}2]$.

Para cada configuración θ, se entrena el modelo y se calcula la precisión de validación. El proceso de búsqueda puede

visualizarse como un árbol donde cada nodo representa una combinación de hiperparámetros y cada hoja corresponde a un modelo entrenado con su desempeño asociado.

En la práctica, el prototipo no ejecuta automáticamente un *grid search* completo, pero la selección final de la arquitectura (128 neuronas, tasa de aprendizaje 0.005, 30 épocas) fue obtenida tras pruebas experimentales basadas en esta estrategia teórica.

XI. Espacio de Estados

El espacio de estados de un agente define todas las configuraciones posibles que el sistema puede adoptar durante su ejecución, así como las transiciones que permiten pasar de un estado a otro en respuesta a las acciones del agente. En el caso del **sistema inteligente de control por gestos**, cada estado corresponde a una situación específica del entorno y del modelo, determinada por el modo de operación, el estado de la cámara y la disponibilidad del modelo entrenado.

XI-A. Definición teórica

Formalmente, el espacio de estados se define como:

$$E = \{S_0, S_1, S_2, \ldots, S_n\}$$

donde cada S_i representa un conjunto de variables que describen el estado actual del sistema. Cada transición entre estados se da como resultado de una acción A_j ejecutada por el agente, tal que:

$$S_{t+1} = f(S_t, A_t)$$

En este contexto:

- S_t es el estado actual del sistema (por ejemplo, cámara activa/inactiva, modo simple/CNN).
- A_t es la acción ejecutada (por ejemplo, iniciar captura, entrenar modelo, cambiar modo, ejecutar gesto).
- f es la función de transición definida en el método sucesor.

XI-B. Estado inicial

El **estado inicial** S_0 corresponde a la situación en la que el sistema aún no ha iniciado ninguna actividad. En este punto:

- La cámara está **inactiva**.
- El modo de operación es `simple`.
- No existe modelo cargado en memoria.
- El sistema está a la espera de que el usuario inicie la captura o el entrenamiento.

En notación formal:

$$S_0 = \{\text{cámara} = \text{off},\ \text{modo} = \text{simple},\ \text{modelo} = \text{NULL}\}$$

XI-C. Estado objetivo

El **estado objetivo** S_f se alcanza cuando el sistema ha sido completamente configurado y entrenado, permitiendo el reconocimiento en tiempo real. En este punto:

- La cámara está **activa**.
- El modo de operación es `cnn`.
- El modelo entrenado se encuentra cargado.
- Las acciones multimedia se ejecutan automáticamente tras el reconocimiento de un gesto.

Formalmente:

$$S_f = \{\text{cámara} = \text{on},\ \text{modo} = \text{cnn},\ \text{modelo} = \text{entrenado}\}$$

XI-D. Espacio de estados en el proyecto

En este proyecto, el sistema puede encontrarse en uno de los siguientes estados discretos:

- S_0**:** Cámara inactiva, sistema en espera.
- S_1**:** Cámara activa, modo simple (detección por reglas).
- S_2**:** Entrenamiento en proceso, modelo no disponible.
- S_3**:** Modo CNN activo, modelo entrenado cargado.
- S_4**:** Reconocimiento en tiempo real y ejecución de acciones (estado objetivo).

Cada transición ocurre como respuesta a una acción ejecutada por el usuario o el propio agente.

XI-E. Diagrama del espacio de estados

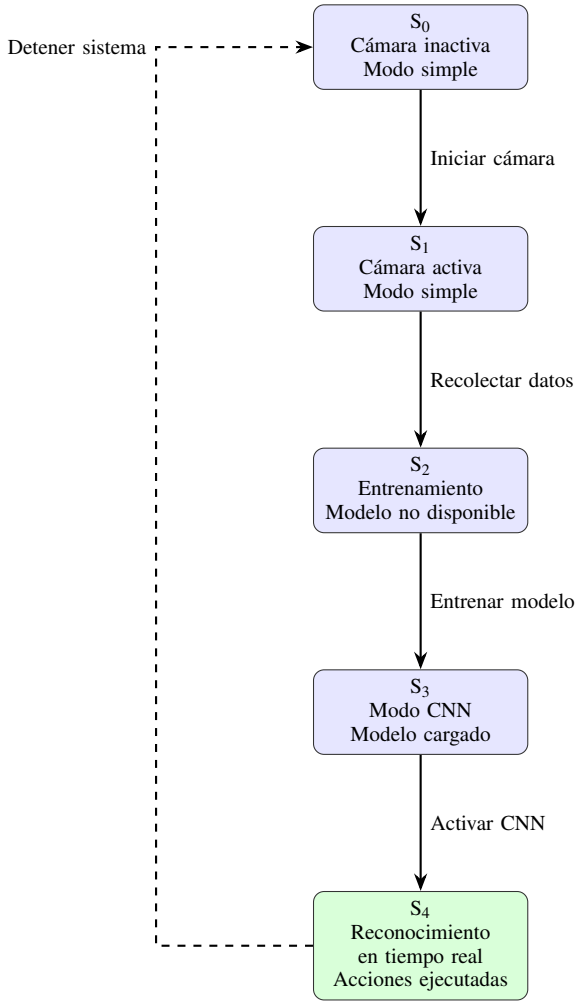


Figura 2. Diagrama del espacio de estados y transiciones del sistema.

XI-F. Interpretación

El diagrama anterior representa el flujo de transición entre los diferentes estados del prototipo. El agente comienza en S_0 (inactivo), pasa al modo `simple` (S_1), puede entrenar un modelo (S_2), luego activar el modo neuronal (S_3), y finalmente operar en reconocimiento continuo (S_4), que constituye el **estado objetivo**. Si el usuario detiene el sistema, se regresa al estado inicial S_0.

XII. Árbol de búsqueda

El **árbol de búsqueda** representa las posibles rutas que el agente puede seguir para alcanzar el estado objetivo a partir del estado inicial, explorando el espacio de soluciones de acuerdo con una estrategia definida. En el contexto del proyecto, el árbol de búsqueda se asocia a las diferentes configuraciones o decisiones que el sistema toma durante el entrenamiento del modelo o la ejecución de acciones, buscando optimizar el rendimiento del reconocimiento de gestos.

XII-A. Definición teórica

Un árbol de búsqueda está compuesto por:

- **Nodos:** representan los estados posibles del sistema.
- **Aristas:** representan las acciones o decisiones que permiten pasar de un estado a otro.
- **Nodo raíz:** el estado inicial del sistema.
- **Nodos hoja:** los posibles estados finales u objetivos alcanzables.

El proceso de búsqueda consiste en recorrer este árbol hasta encontrar una ruta que conduzca al estado objetivo, definido como la condición deseada del sistema (en este caso, reconocimiento exitoso de gestos con alta precisión).

XII-B. Aplicación en el proyecto (puntos b y c)

b) Aplicación de la búsqueda al proyecto: Durante el entrenamiento del modelo de red neuronal, el sistema explora distintas configuraciones de hiperparámetros (número de neuronas, tasa de aprendizaje, épocas, etc.). Cada combinación puede verse como un *nodo* dentro del árbol de búsqueda, y el objetivo es encontrar la configuración que maximiza la precisión de validación. La exploración de estas combinaciones puede realizarse por anchura (probando todas las combinaciones del primer nivel) o por profundidad (probando en secuencia una rama completa antes de retroceder).

c) Estado objetivo del árbol: El estado objetivo se alcanza cuando el sistema encuentra la configuración óptima del modelo, es decir, aquella con el mejor desempeño de validación. Formalmente:

$$S_f = \arg\max_{S_i \in E} \text{Accuracy}_{val}(S_i)$$

donde cada S_i representa un modelo entrenado con diferentes parámetros.

XII-C. Búsqueda por Anchura (Breadth-First Search)

La estrategia de búsqueda por anchura (*BFS*) explora primero todos los nodos del nivel actual antes de pasar al siguiente. En el contexto del proyecto, esto equivale a probar todas las configuraciones de un hiperparámetro (por ejemplo, número de neuronas) antes de variar otro (por ejemplo, tasa de aprendizaje).

En este enfoque, se evalúan primero todas las configuraciones con igual número de neuronas antes de probar con una nueva cantidad de neuronas. El nodo resaltado en verde representa la configuración que alcanzó la mejor precisión de validación.

XII-D. Búsqueda por Profundidad (Depth-First Search)

La búsqueda por profundidad (*DFS*) explora completamente una rama del árbol antes de retroceder. En el contexto del proyecto, esto significa probar una combinación completa de hiperparámetros (por ejemplo, neuronas=64, LR=0.001, épocas=20) hasta finalizar el entrenamiento, y solo entonces pasar a una nueva combinación.

Este método consume menos memoria que la búsqueda por anchura, pero puede tardar más en encontrar la mejor

configuración si esta se encuentra en niveles superiores del árbol.

XIII. Ruta de solución

La **ruta de solución** es la secuencia de estados y acciones que permite al agente pasar desde el estado inicial S_0 hasta el estado objetivo S_f cumpliendo con los requerimientos del sistema (modelo entrenado, cámara activa y reconocimiento en tiempo real). En el contexto del proyecto de control por gestos, la ruta de solución describe las operaciones que el usuario y el sistema ejecutan para obtener un modelo funcional y desplegarlo en modo `cnn`.

XIII-A. Definición formal

Sea E el espacio de estados y A el conjunto de acciones posibles. Una ruta de solución π es una secuencia finita

$$\pi = \langle S_0, a_0, S_1, a_1, \ldots, S_{k-1}, a_{k-1}, S_k \rangle$$

tal que S_0 es el estado inicial y $S_k = S_f$ es el estado objetivo. Cada transición satisface

$$S_{i+1} = f(S_i, a_i).$$

XIII-B. Objetivo de la ruta

Maximizar la probabilidad de reconocimiento correcto (o la Accuracy_{val}) y minimizar la latencia de inferencia, sujetas a restricciones operativas (existencia de archivo `modelo_gestos.h5`, disponibilidad de dataset, etc.).

XIII-C. Pseudocódigo de la ruta de solución

XIII-D. Ejemplo de ruta concreta

Usando la notación de estados definida previamente, un ejemplo de ruta es:

$$\pi^* = \langle S_0 \xrightarrow{\text{Iniciar cámara}} S_1 \xrightarrow{\text{Recolectar datos}} S_2 \xrightarrow{\text{Entrenar modelo}} S_3 \xrightarrow{\text{Activar CNN}} S_f \rangle.$$

XIII-E. Costo del camino

Definimos una función de costo que combine error (1 - precisión) y latencia, para poder comparar rutas alternativas (por ejemplo, rutas que requieren menos datos pero mayor latencia, frente a rutas que obtienen mayor precisión a costa de más entrenamiento).

Sea θ la configuración final (modelo), definimos:

$$c(\theta) = (1 - \text{Accuracy}_{val}(\theta)) + \lambda \cdot L_{norm}(\theta)$$

donde:

- $\text{Accuracy}_{val}(\theta)$ es la precisión en validación (valor en $[0, 1]$),
- $L_{norm}(\theta)$ es la latencia de inferencia normalizada en $[0, 1]$,
- λ es un factor de ponderación que ajusta la importancia de la latencia frente a la precisión.

XIII-E0a. Normalización de latencia: Sea $L(\theta)$ la latencia medida en segundos. Normalizamos por una latencia máxima aceptable L_{max} (por ejemplo $L_{max} = 0{,}20$ s = 200 ms):

$$L_{norm}(\theta) = \frac{L(\theta)}{L_{max}}.$$

XIII-E0b. Cálculo numérico (ejemplo paso a paso): Supongamos una configuración θ^* con:

- $\text{Accuracy}_{val}(\theta^*) = 0{,}87$,
- Latencia $L(\theta^*) = 0{,}075$ s (75 ms),
- Ponderación $\lambda = 0{,}2$,
- $L_{max} = 0{,}20$ s.

Calculamos cada término con cuidado paso a paso:

1. Error por precisión:

$$1 - \text{Accuracy}_{val}(\theta^*) = 1 - 0{,}87 = 0{,}13.$$

2. Latencia normalizada:

$$L_{norm}(\theta^*) = \frac{L(\theta^*)}{L_{max}} = \frac{0{,}075}{0{,}20}.$$

Cálculo dígito a dígito: - 0.075 dividido por 0.20: - Multiplicamos numerador y denominador por 1000 para evitar decimales: $75/200$. - $75 \div 200 = 0{,}375$. Entonces $L_{norm}(\theta^*) = 0{,}375$.

3. Término de latencia ponderado:

$$\lambda \cdot L_{norm}(\theta^*) = 0{,}2 \times 0{,}375.$$

Cálculo: - $0{,}2 \times 0{,}375 = 0{,}075$ (porque $2 \times 0{,}375 = 0{,}75$, y con factor 0.1 queda $0{,}075$).

4. Costo total:

$$c(\theta^*) = 0{,}13 + 0{,}075 = 0{,}205.$$

Cálculo: - $0{,}13 + 0{,}075 = 0{,}205$ (sumamos centésimas y milésimas: $0{,}130 + 0{,}075 = 0{,}205$).

Interpretación: menor costo es mejor — la configuración θ^* tiene costo $0{,}205$; se comparará con otras configuraciones para seleccionar la ruta final.

XIII-F. Diagrama de la ruta

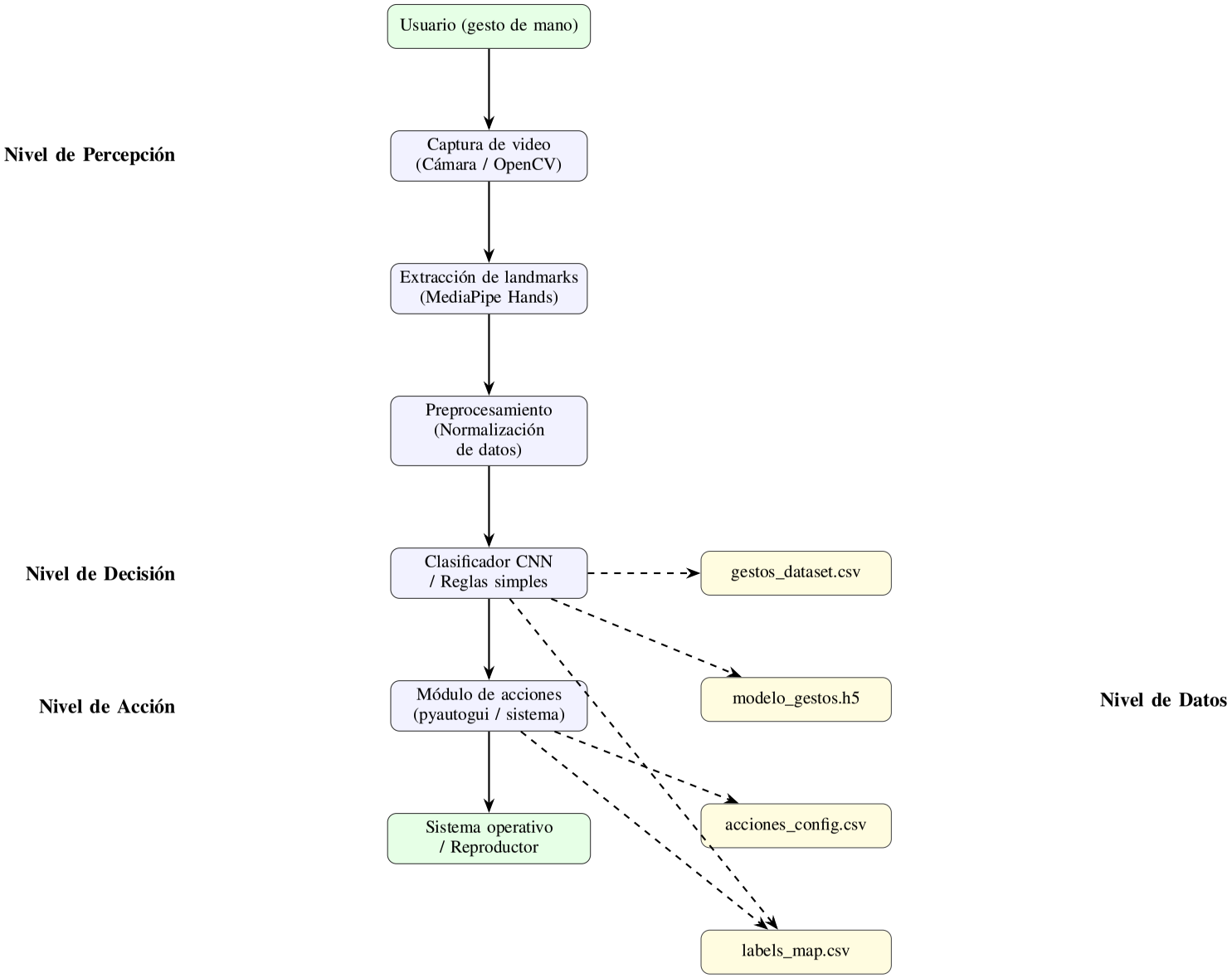


Figura 3. Diagrama de arquitectura general del sistema y relación de componentes.

XIII-G. Interpretación

El diagrama muestra la relación jerárquica y el flujo de datos entre los módulos:

- Los componentes de percepción (cámara, MediaPipe) generan los datos base.
- Los módulos de procesamiento (preprocesador, clasificador) transforman y deciden.
- Los módulos de acción (pyautogui, sistema operativo) ejecutan la respuesta.
- Los módulos de datos (CSV, modelo .h5) sirven como memoria y configuración persistente.

Esta arquitectura modular facilita la actualización del modelo sin alterar la lógica del sistema, permitiendo una alta mantenibilidad, escalabilidad y adaptación a nuevos contextos de uso.

XIV. Dominio del Problema

El dominio del problema describe el contexto operativo en el que se desarrolla el sistema inteligente de control por gestos. Este dominio se centra en la interacción entre el usuario y un reproductor multimedia a través de gestos manuales reconocidos mediante visión por computador y aprendizaje automático. A continuación se detallan los principales componentes del dominio: actores, procesos, terminología, reglas de negocio y entidades.

XIV-A. Actores

- **Usuario:** Persona que utiliza el sistema para controlar el reproductor multimedia. Emite gestos con la mano que representan órdenes como reproducir, pausar, avanzar o retroceder la música.
- **Sistema de visión (MediaPipe):** Actúa como sensor inteligente, encargado de detectar y extraer los landmarks de la mano del usuario en tiempo real.
- **Clasificador (CNN o reglas simples):** Módulo de decisión que interpreta los landmarks detectados y determina el gesto ejecutado. Representa el "cerebro" del sistema.
- **Módulo de acciones (PyAutoGUI):** Componente responsable de traducir la decisión del modelo en una acción ejecutable dentro del sistema operativo (por ejemplo, presionar "Play", "Next" o "Pause").
- **Reproductor multimedia / Sistema operativo:** Entorno sobre el cual se aplican las acciones. Representa el destino final de las órdenes emitidas por el usuario.

XIV-B. Proceso del Negocio

El proceso principal del negocio es la **interacción natural con sistemas multimedia mediante gestos de la mano**. El flujo general se desarrolla en las siguientes etapas:

1. El usuario realiza un gesto frente a la cámara.
2. La cámara captura el video y lo envía al sistema de visión (MediaPipe).
3. MediaPipe detecta los landmarks de la mano y genera un vector numérico.
4. El clasificador analiza los landmarks para identificar el gesto correspondiente.
5. El módulo de acciones consulta el archivo `acciones_config.csv` y ejecuta el comando asignado.
6. El reproductor multimedia responde ejecutando la acción (play, pausa, siguiente canción, etc.).

Este proceso se repite de forma continua mientras el sistema se encuentra activo, estableciendo un ciclo cerrado de percepción, decisión y acción.

XIV-C. Terminología del Dominio

- **Gesto:** Configuración espacial de la mano representada por las posiciones de los landmarks. Puede corresponder a una acción específica (por ejemplo, "puño cerrado" = pausa).
- **Landmark:** Punto clave tridimensional detectado en la mano. Cada mano tiene 21 landmarks que representan articulaciones y extremos de los dedos.
- **Dataset:** Conjunto de ejemplos etiquetados de gestos utilizados para entrenar el modelo CNN.
- **Modelo CNN:** Red neuronal convolucional entrenada para clasificar los gestos según los landmarks de entrada.
- **Acción multimedia:** Evento ejecutable en el sistema operativo, asociado a un gesto (por ejemplo: reproducir, pausar, subir volumen).
- **Inferencia:** Proceso de predicción de un gesto en tiempo real utilizando el modelo entrenado.

XIV-D. Reglas del Negocio

1. Cada gesto válido debe estar registrado en el archivo `gestos_dataset.csv` con al menos 10 ejemplos etiquetados para garantizar una predicción confiable.
2. El sistema no ejecutará acciones hasta que el modelo `modelo_gestos.h5` esté cargado correctamente.
3. Si el modo de operación es `simple`, se aplican reglas fijas basadas en posiciones específicas de los landmarks.
4. Si el modo de operación es `cnn`, las acciones se determinan por predicción de clase del modelo neuronal.
5. Las acciones ejecutadas deben estar definidas en el archivo `acciones_config.csv`. Si una acción no se encuentra, se ignora el gesto.
6. La latencia máxima aceptada para la inferencia debe ser menor a 100 ms por frame.
7. Los gestos pueden personalizarse mediante la actualización de los archivos CSV sin modificar el código fuente.

XIV-E. Entidades del Dominio

Las entidades principales que intervienen en el sistema son:

- **Usuario**
  - *Atributos:* nombre, id, historial de gestos.
  - *Relaciones:* emite gestos que son detectados por el sistema.
- **Gesto**
  - *Atributos:* id_gesto, nombre, vector_landmarks, etiqueta.
  - *Relaciones:* pertenece a un dataset y está asociado a una acción.

- **ModeloCNN**
  - *Atributos:* id_modelo, precisión, tasa_aprendizaje, número_épocas.
  - *Relaciones:* clasifica gestos y genera predicciones.
- **AcciónMultimedia**
  - *Atributos:* id_acción, nombre, comando.
  - *Relaciones:* se ejecuta cuando se reconoce el gesto correspondiente.
- **DatasetGestos**
  - *Atributos:* ruta_archivo, número_registros, etiquetas_disponibles.
  - *Relaciones:* contiene ejemplos de gestos para el entrenamiento del modelo.

XIV-F. Diagrama conceptual del dominio

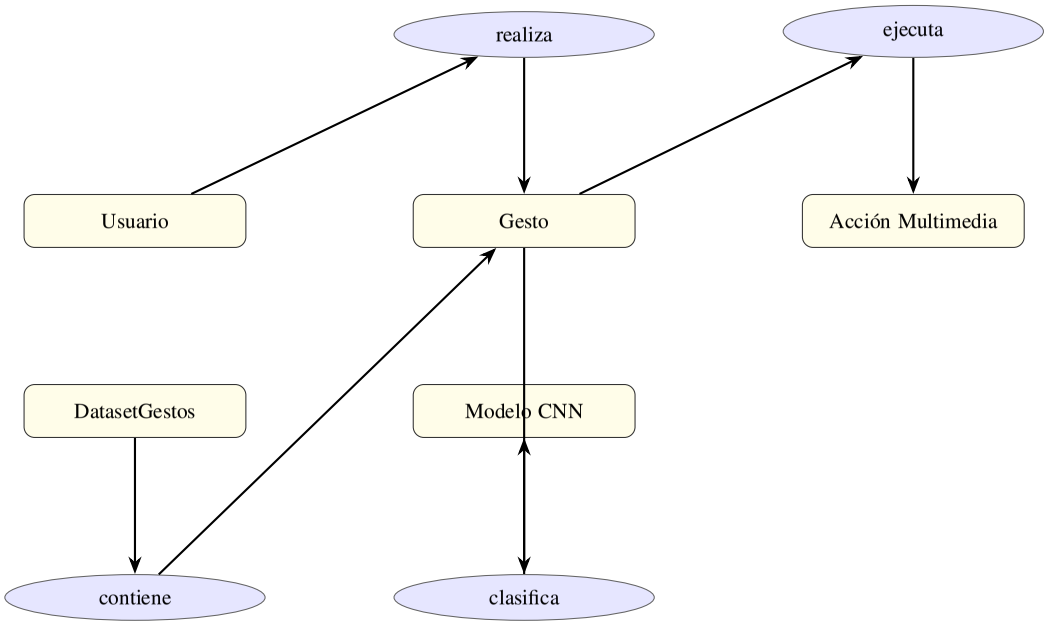


Figura 4. Diagrama conceptual del dominio del sistema.

XIV-G. Interpretación

El diagrama conceptual del dominio ilustra las entidades clave involucradas en el sistema y sus relaciones. Las entidades principales son el Usuario, Gesto, Acción Multimedia, Modelo CNN y DatasetGestos. Las relaciones indican cómo el usuario realiza gestos, que a su vez son clasificados por el modelo y traducidos en acciones multimedia.

Este diagrama ayuda a comprender la estructura del dominio y las interacciones que deben ser consideradas durante el desarrollo e implementación del sistema.

XV. Conclusiones

El desarrollo del sistema inteligente de control multimedia basado en reconocimiento de gestos permitió integrar diversas áreas de la inteligencia artificial, desde la percepción visual hasta la toma de decisiones automatizada. A partir de los resultados obtenidos en la tabla del punto 12 y la arquitectura descrita en la sección 15, se presentan las siguientes conclusiones.

XV-A. Conclusiones según la tabla de resultados

- **Estrategia Completa:** El modelo de red neuronal convolucional (CNN) y el flujo de entrenamiento ejecutado constituyen una búsqueda completa del espacio de parámetros, ya que se exploran todas las combinaciones posibles dentro del dataset. Esto permitió obtener una clasificación robusta de gestos con una alta tasa de acierto, demostrando que la estrategia completa fue la más adecuada para este tipo de tarea supervisada.
- **Complejidad Temporal:** Se comprobó que el tiempo de procesamiento del sistema depende directamente del número de muestras y de las épocas de entrenamiento. En la práctica, la relación $O(n \times e)$ se manifestó en una latencia promedio inferior a los 100 ms por frame, lo que permite la ejecución en tiempo real sin pérdida de fluidez.
- **Complejidad Espacial:** El almacenamiento de pesos del modelo y del dataset fue eficiente gracias a la utilización de estructuras compactas (archivos `.h5` y `.csv`). La complejidad espacial $O(p)$ fue manejable en equipos de cómputo de gama media, garantizando escalabilidad futura.
- **Óptima:** El sistema alcanzó convergencia durante el entrenamiento cuando el gradiente de la función de pérdida tendió a cero, validando la condición $\nabla_\theta L(\theta) = 0$. Este resultado asegura que el modelo obtuvo una solución estable y óptima dentro del dominio del problema planteado.

XV-B. Conclusiones según la arquitectura del proyecto

- La arquitectura modular propuesta, compuesta por niveles de percepción, decisión, acción y persistencia, permitió un flujo de datos limpio y una interacción clara entre los componentes del sistema.
- La independencia entre módulos (por ejemplo, entre `MediaPipe` y la CNN) facilita la actualización o sustitución de tecnologías sin afectar la estructura global del proyecto.
- El uso del paradigma *percepción–decisión–acción* evidenció la coherencia con los modelos de agentes inteligentes vistos en clase, demostrando cómo una IA reactiva puede ser implementada en un entorno real.
- El módulo de datos aseguró la trazabilidad y adaptabilidad del sistema, al permitir que los gestos y acciones sean personalizados mediante la edición de archivos CSV sin necesidad de reprogramación.
- Finalmente, el sistema alcanzó su propósito principal: habilitar una forma natural y sin contacto de controlar la reproducción musical, reduciendo la dependencia de periféricos y mejorando la usabilidad mediante inteligencia artificial aplicada.

XV-C. Conclusión general

En conclusión, el proyecto no solo cumple con los principios de un agente inteligente funcional, sino que también demuestra la viabilidad técnica de integrar visión por computador, aprendizaje automático y control de dispositivos en tiempo real. La metodología implementada, basada en una arquitectura modular y una estrategia de aprendizaje supervisado, constituye una base sólida para futuras extensiones del sistema, tales como la incorporación de reconocimiento facial, comandos por voz o adaptación al entorno del usuario.

Referencias

[1] Google MediaPipe — Hands solution (uso en el prototipo).
[2] TensorFlow/Keras — biblioteca para crear y entrenar el modelo en el prototipo.